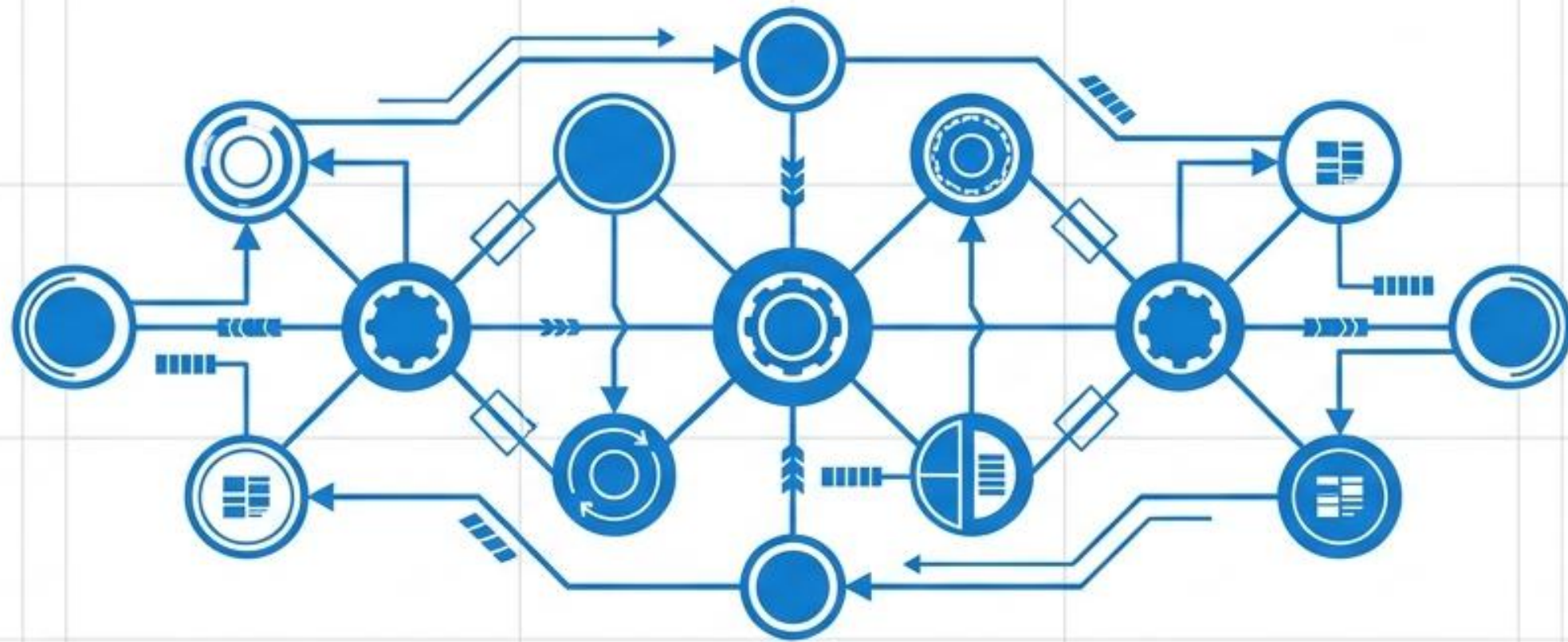


Loading Knowledge Graph Data

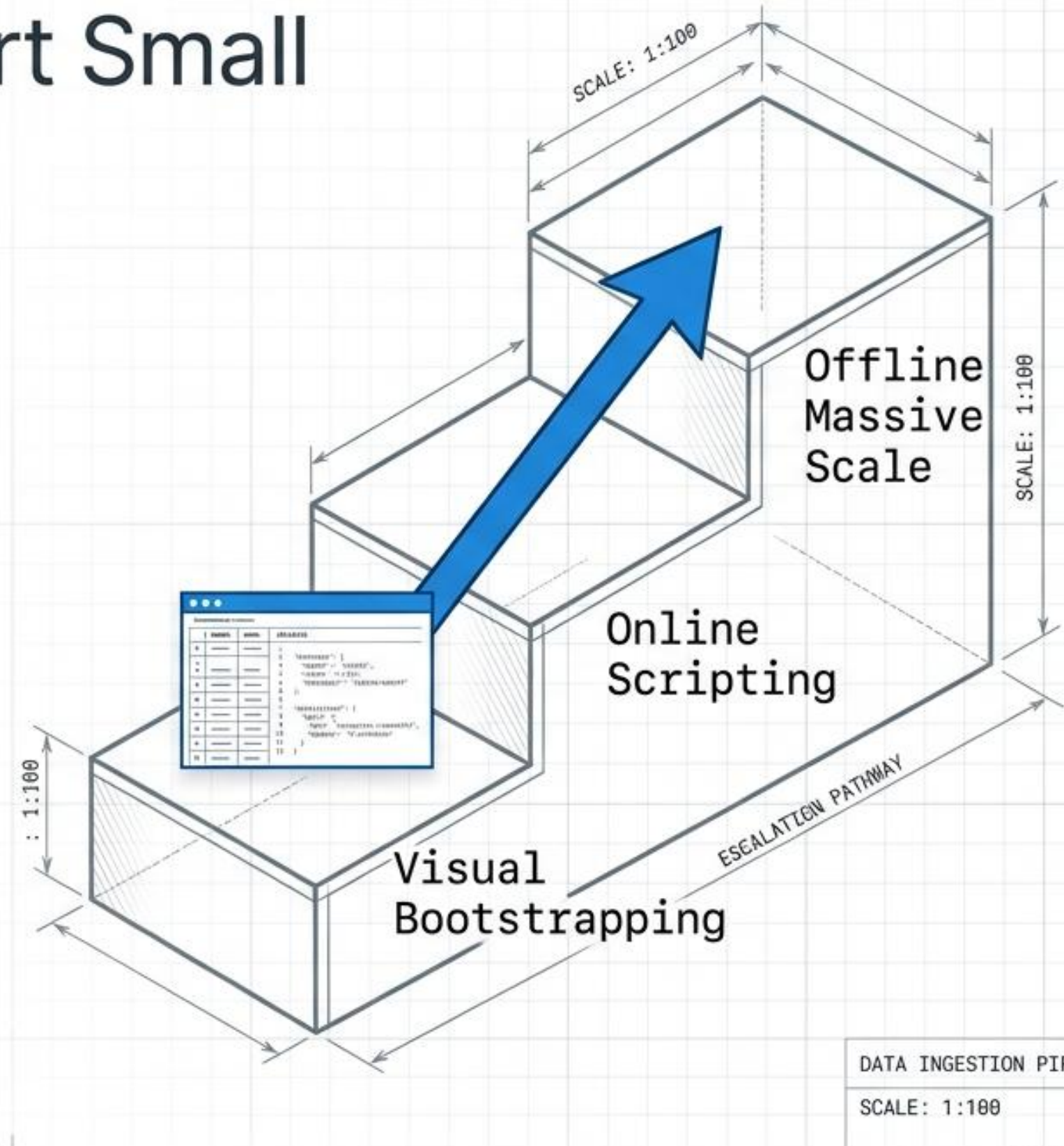
Architecting Data Logistics Pipelines for Scale



dr Zbigniew Galar

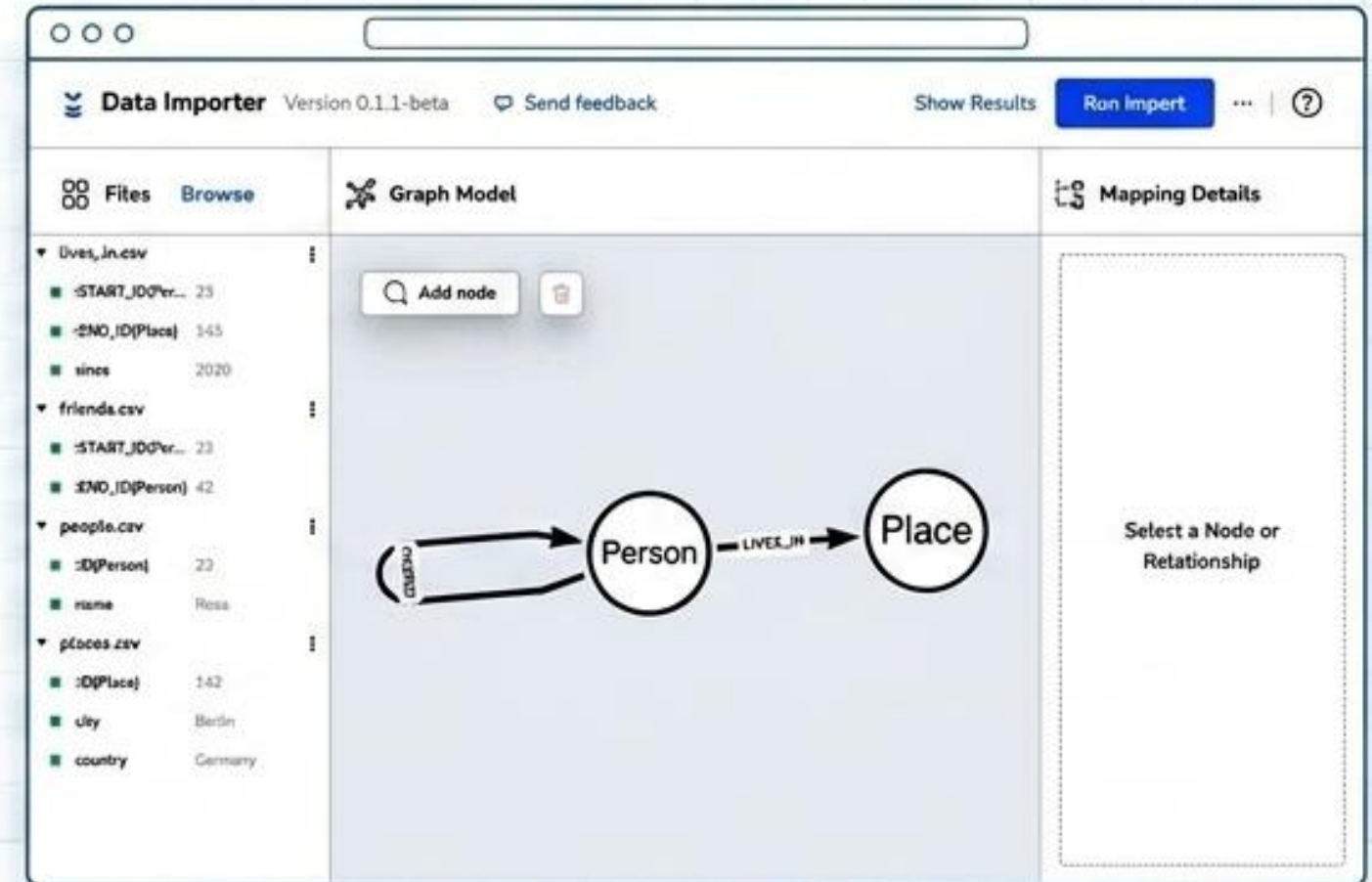
The Universal Rule: Start Small

- Bulk imports span the entire lifecycle of a knowledge graph.
- Success requires taking a small, representative slice of data first.
- Import and verify the model locally before executing a massive ingestion job to save hours of frustration.



Level 1: Visual Bootstrapping

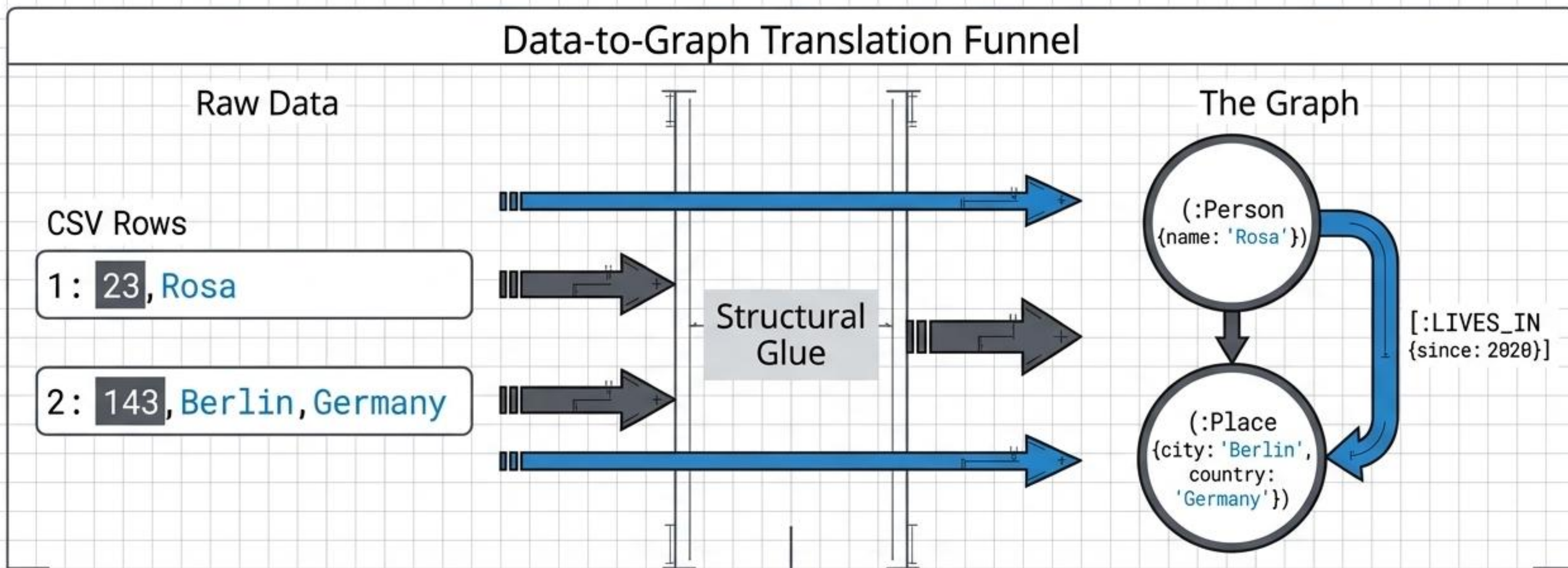
- The Neo4j Data Importer simplifies early-stage prototyping with a graphical interface.
- Developers draw a domain model and directly map ready-to-use CSV files onto visual nodes and relationships.



Metrics Panel	
Target:	Beginners to Experienced Devs
Database Status:	Online
Input:	Local CSV files

DATA INGESTION PIPELINE
SCALE: 1:100

Translating Flat Files to Graph Topologies



Unique IDs link the graph together during import. They act purely as structural scaffolding and can be expunged later, leaving only the cleanly mapped properties and relationships.

The Anatomy of Auto-Generated Cypher

The visual mapper writes production-ready Cypher underneath that can be exported, modified, and hosted in source control.

Import Results

Time Taken: 00:00:00

● Import complete

Person		people.csv					
Time Taken	File Size	File Rows	Nodes Created	Properties Set	Labels Added	Query Count	Query Time
00:00:00	42 B	3	0	3	0	1	00:00:00

Transforms lists of CSV values into individual rows fed into subsequent statements.

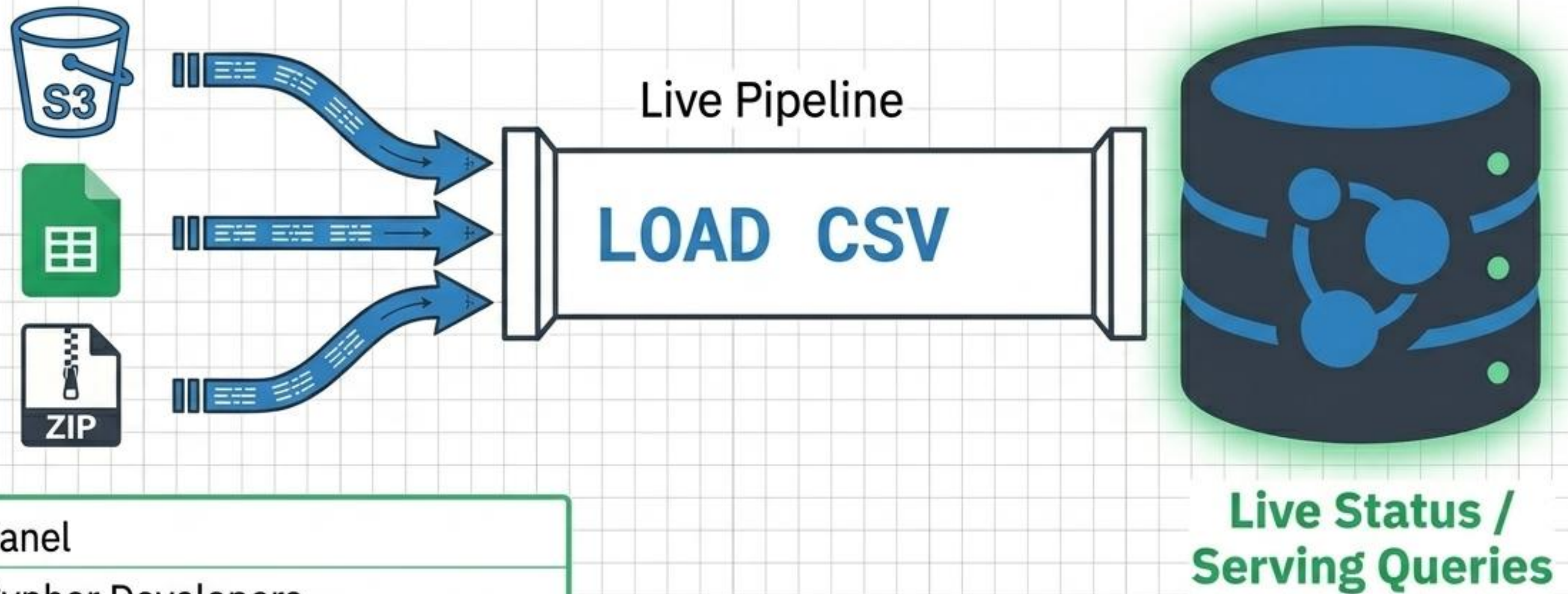
```
UNWIND $nodeRecords AS nodeRecord  
MERGE (n:Person {ID: nodeRecord.ID})  
SET n.name = nodeRecord.name
```

Tolerates the absence of data without failing.

Acts like MATCH if a record exists, and CREATE if it does not—preventing duplicate creation.

Level 2: Online Scripting with LOAD CSV

For continuous integration, LOAD CSV imports data from remote web addresses or filesystems into a live, querying database. It handles compressed payloads and executes natively within Cypher.



Metrics Panel

Target: Cypher Developers

Database Status: Online

Input: Local, S3, Web, Compressed CSV

DATA INGESTION PIPELINE

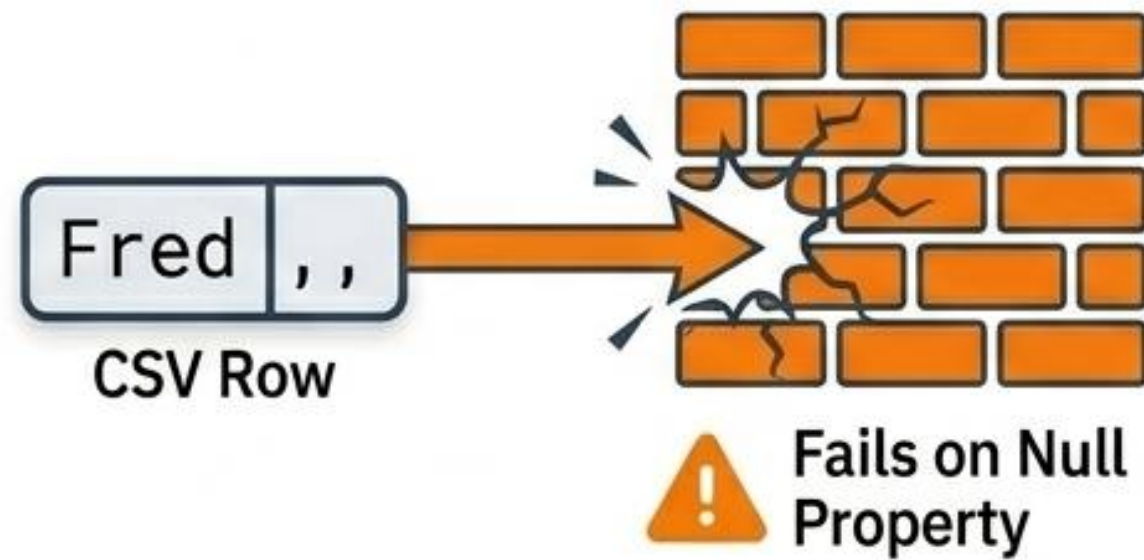
SCALE: 1:100

Managing Irregular Data in the Real World

Attempting to write null properties directly during node creation will cause the operation to fail. Relying on SET operations post-creation ensures data flows cleanly even when columns are empty.

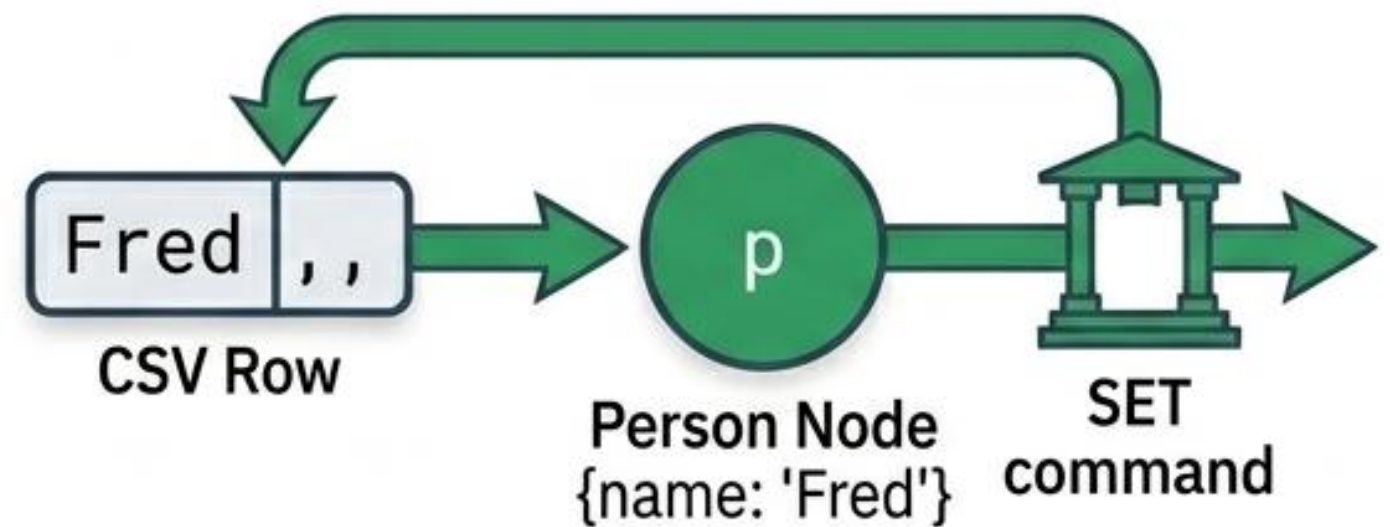
The Naive Approach

```
MERGE (p:Person {name: line.name,  
age: line.age})
```



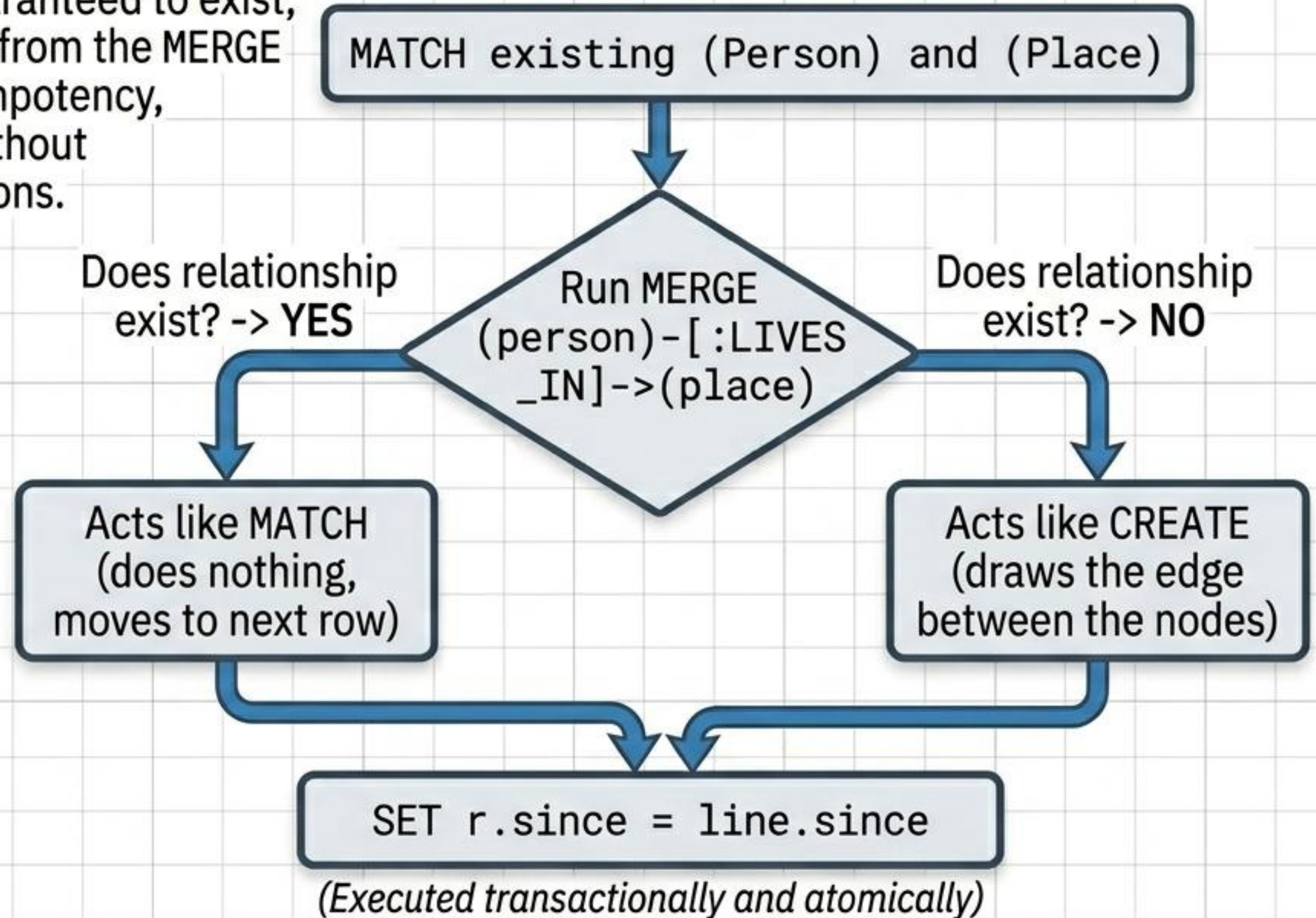
The Resilient Approach

```
MERGE (p:Person {name: line.name})  
SET p.age = line.age
```



Resolving Duplicate Relationships Safely

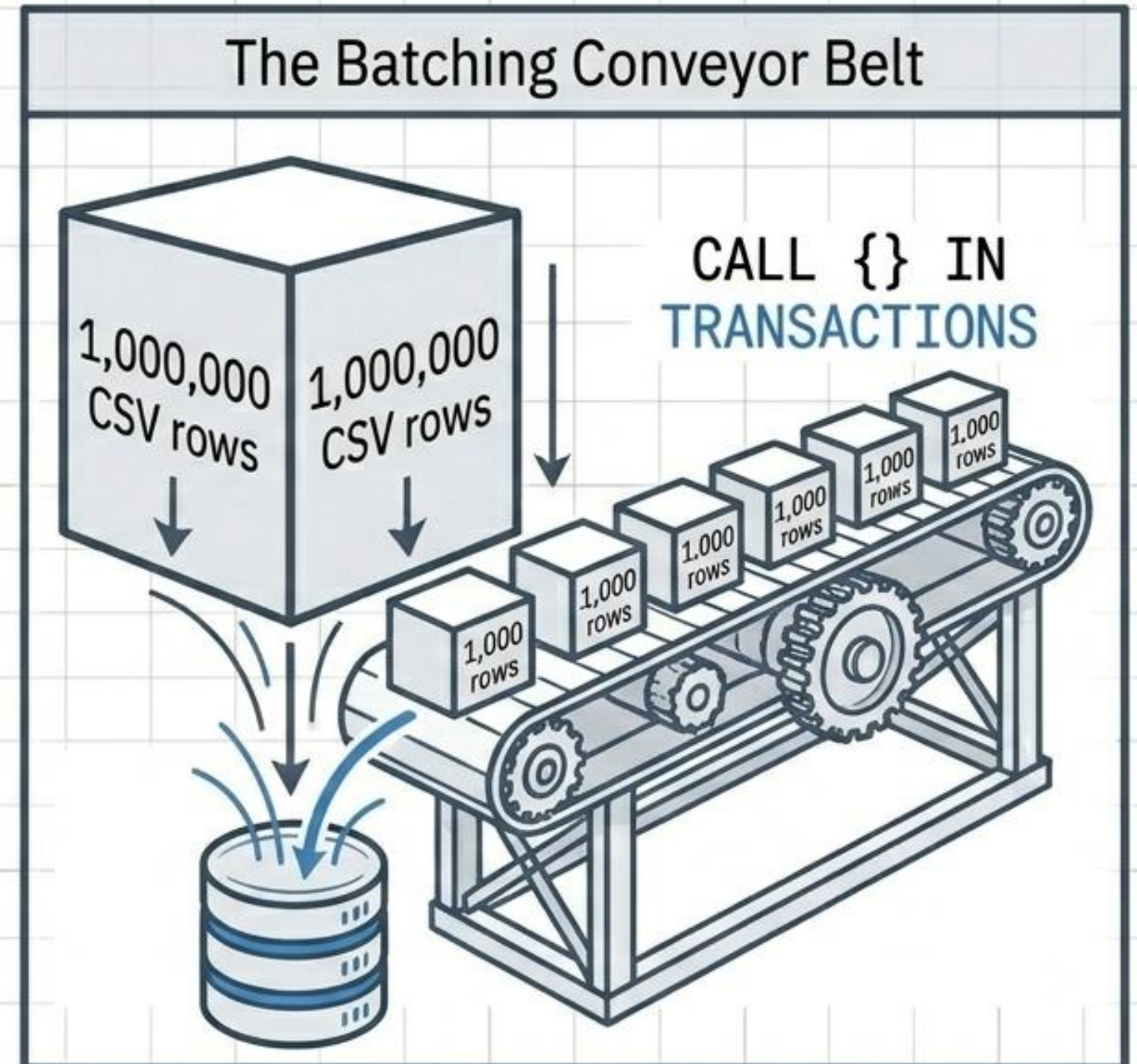
When linking entities that are guaranteed to exist, separate the MATCH of the nodes from the MERGE of the edge. This guarantees idempotency, ensuring scripts can be re-run without compounding duplicate connections.



Preventing Memory Overflow with Batching

Segmenting imports into manageable batches ensures smooth ingestion without interrupting active queries. Pushing millions of records in a single transaction will overwhelm memory constraints.

```
CALL {  
  WITH line  
  MERGE (p:Person {name: line.name})  
} IN TRANSACTIONS OF 1000 ROWS
```



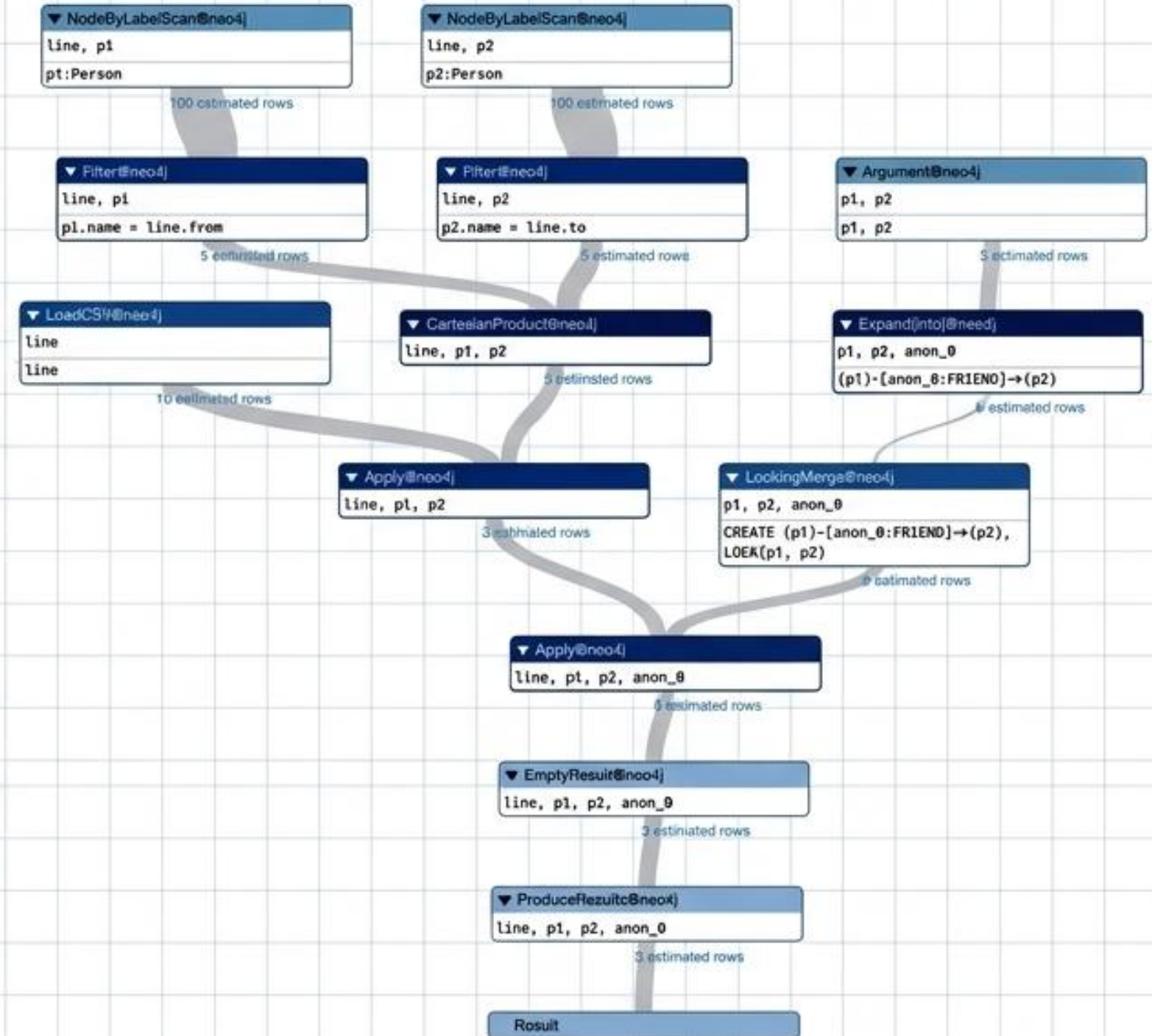
Profiling and Tuning the Pipeline

LOAD CSV accepts standard Cypher diagnostic prefixes like EXPLAIN and PROFILE to analyze performance before execution.

Diagnostic Warnings

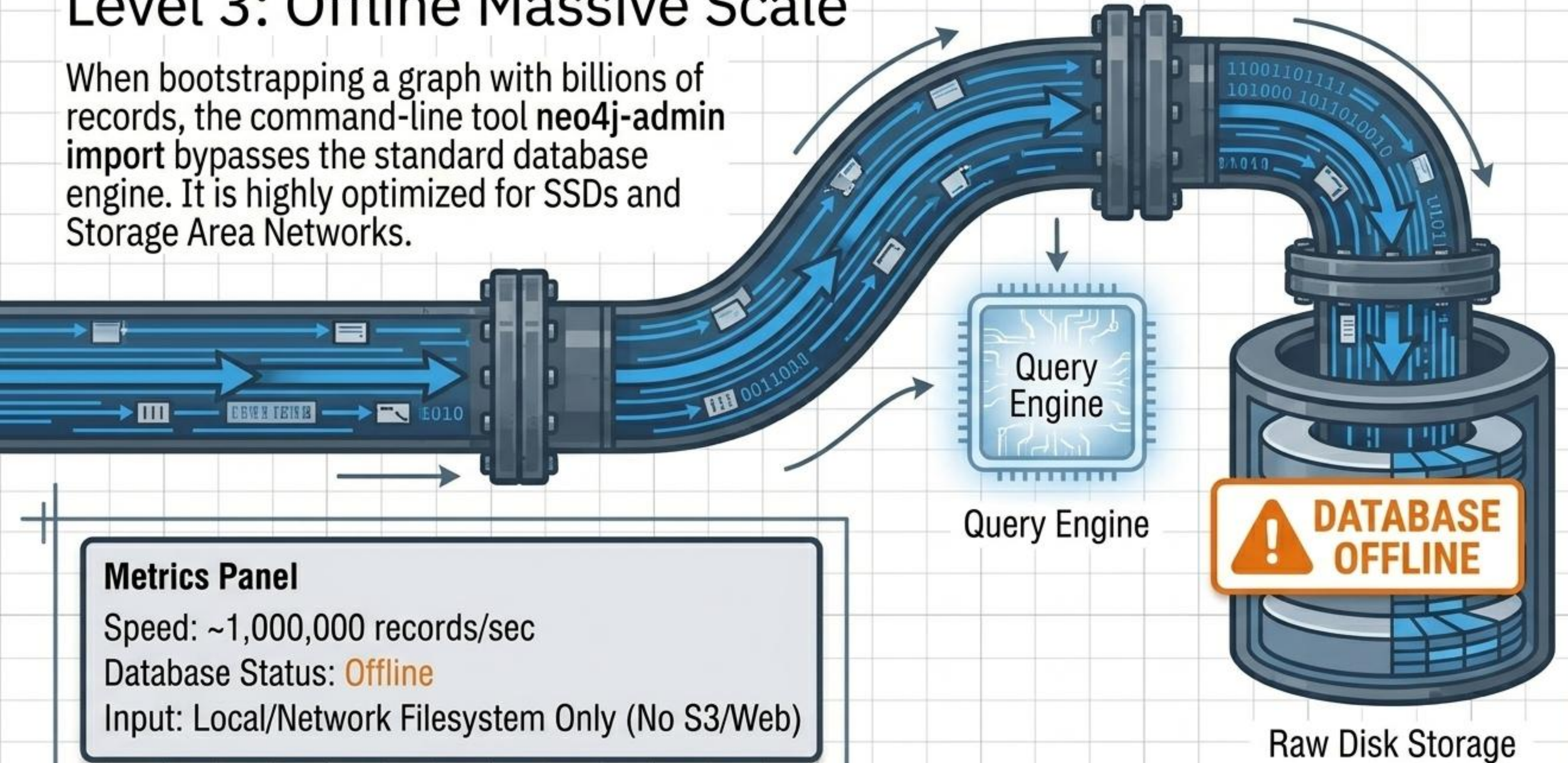
Cartesian Products: Acceptable only if fed by a filter that heavily reduces matching nodes.

Eager Operators: Pull in all data immediately, creating severe pipeline choke points. Eliminate these to maintain high throughput.



Level 3: Offline Massive Scale

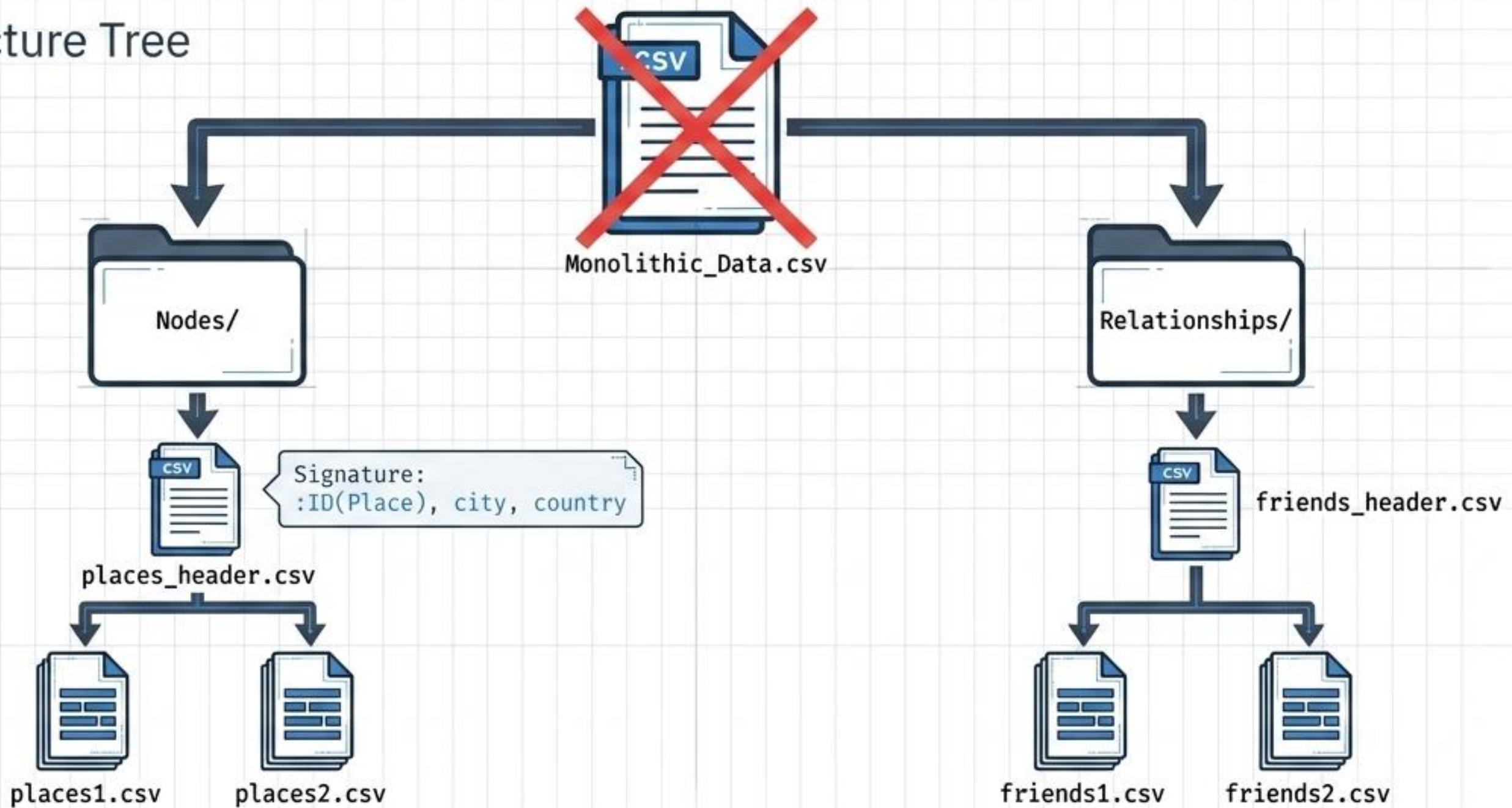
When bootstrapping a graph with billions of records, the command-line tool **neo4j-admin import** bypasses the standard database engine. It is highly optimized for SSDs and Storage Area Networks.



Optimizing File Architecture for Speed

neo4j-admin import requires nodes and relationships to be separated. Splitting headers into their own isolated files preserves text-editing sanity, preventing developers from opening multi-gigabyte data files just to edit a column name.

File Architecture Tree



The Anatomy of a High-Speed Ingestion Command

Multiline execution requires careful syntax.

```
bin/neo4j-admin import \  
  --nodes=Person=import/people.csv \  
  --relationships=FRIEND=import/friends_header.csv,import/friends1.csv \  
  
```

The tool logically treats split header and data arrays as a single continuous unit.

Execution Constraint: The tool is incredibly fast but strictly non-resumable. A single Ctrl-C during a multi-hour import requires starting completely over.

The Master Importer Decision Matrix

Selecting the right ingestion architecture depends entirely on scale constraints, database uptime requirements, and the current phase of the knowledge graph lifecycle.

	Neo4j Data Importer	LOAD CSV	neo4j-admin import
Primary Use	Prototyping & Mapping	Continuous Updates	Massive Bootstrapping
DB Status	 Online	 Online	 Offline
Speed/Scale	Thousands	Millions (Batched)	Billions (1M/sec)
Data Format	CSV Files	Web, S3, CSV, Zipped	Local/Network Filesystem CSV
Data Quality	Requires Clean Data	Handles Irregular/Messy	Requires Clean Characters

Architecting the Complete Data Lifecycle

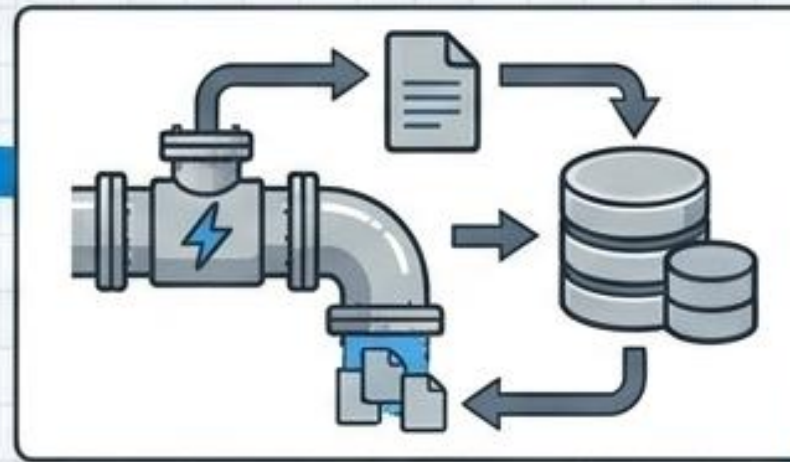
Bulk imports are not a one-time event. True mastery lies in utilizing all three architectures: starting small to validate logic, bootstrapping offline for historical mass, and deploying transactional scripts to maintain a living knowledge graph.

Phase 1: Start Small



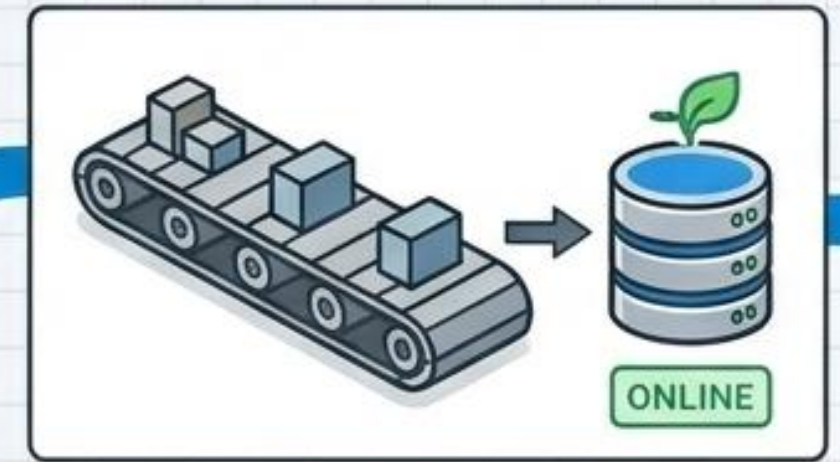
Importer tests 100 rows to validate the schema.

Phase 2: Massive Bootstrap



neo4j-admin loads the historical 10-year dataset offline.

Phase 3: Continuous Operation



LOAD CSV runs nightly scripts to append daily changes to the live graph.